



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE  
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN  
IIC2413 - BASES DE DATOS

# Solución Interrogación 2

Fecha: 19 de junio de 2026

1<sup>er</sup> semestre 2026 - Profesores: Eduardo Bustos - Christian Álvarez

---

## Si Ud. no lee instrucciones, lea al menos estas INSTRUCCIONES

- a) La prueba tiene 4 preguntas, cada una vale 15 puntos y una pregunta Bonus de 5 puntos. **Duración 2 horas.**
- b) Lea toda la prueba primero, las preguntas NO ESTÁN en orden de dificultad
- c) Al finalizar la prueba debe digitalizar cada pregunta en formato PDF VERTICAL y subirlas a Canvas **correctamente.**
- d) Cada plana de respuesta debe contener su Nombre Completo y Número de Lista en la parte superior.
- e) No mezcle respuestas de diferentes preguntas en la misma plana. Si puede responder una pregunta en una plana y otra en la otra plana de la misma hoja.
- f) Debe firmar la lista de asistencia.
- g) No seguir esta instrucción dificulta el trabajo de evaluación y por ende el tiempo de entrega de las notas, además los expone a un descuento de hasta 5 décimas de la nota final.

## Pregunta 1 - ACID, Transacciones y Recuperación de Fallas

### a) Propiedades ACID (4 pts)

| Propiedad    | Definición                                                                                                         | Módulo responsable     |
|--------------|--------------------------------------------------------------------------------------------------------------------|------------------------|
| Atomicidad   | O se ejecutan todas las operaciones de la transacción, o no se ejecuta ninguna.                                    | Log y Recovery Manager |
| Consistencia | Cada transacción preserva la consistencia de la BD (restricciones de integridad, llaves primarias/foráneas, etc.). | Transaction Manager    |
| Aislamiento  | Cada transacción se ejecuta como si lo hiciera sola, de forma aislada de las demás.                                | Transaction Manager    |
| Durabilidad  | Los cambios de una transacción confirmada son permanentes, sin importar fallas posteriores.                        | Log y Recovery Manager |

### b) Redo Logging (6 pts)

```

<START T1>
<START T2>
<T1, A, 100>
<T2, B, 200>
<T1, C, 150>
<COMMIT T2>
<START T3>
<T3, D, 50>
<COMMIT T1>
<T2, END>
* FALLA AQUI

```

#### i) ¿Cuáles transacciones se rehacen? (3 pts)

Regla de Redo Logging: solo se rehacen transacciones que tienen <COMMIT T> en el log.

- **T1: se rehace** → tiene <COMMIT T1>.
- **T2: se ignora** → ya tiene <T2, END>; esto significa que su redo ya fue aplicado y confirmado en disco, no hay nada pendiente.
- **T3: no se rehace** → no tiene <COMMIT T3> en el log, por lo que sus cambios nunca se garantizaron y se descartan (D no se actualiza).

#### ii) Orden de aplicación y valores finales (3 pts)

El recovery se procesa **de inicio a fin** del log. Se aplican en orden las escrituras de las transacciones que sí tienen COMMIT:

- a)  $\langle T1, A, 100 \rangle \rightarrow$  T1 tiene COMMIT más adelante  $\rightarrow$  se aplicará  $A = 100$ .
- b)  $\langle T2, B, 200 \rangle \rightarrow$  T2 tiene COMMIT y END  $\rightarrow$  no es necesario aplicar.
- c)  $\langle T1, C, 150 \rangle \rightarrow$  T1 tiene COMMIT más adelante  $\rightarrow$  se aplicará  $C = 150$ .
- d)  $\langle T3, D, 50 \rangle \rightarrow$  T3 **no** tiene COMMIT  $\rightarrow$  no se aplica.
- e) Procesando  $\langle \text{COMMIT } T1 \rangle \rightarrow$  se aplica:  $A = 100$  y  $C = 150$ .

**Valores finales en disco:**

$A = 100 \quad B = 200 \quad C = 150 \quad D = \text{sin modificar (valor anterior)}$

### c) Undo Logging con Nonquiescent Checkpoint (5 pts)

```

<START T1>
<T1, a, 5>
<START T2>
<T2, b, 10>
<START CKPT (T1, T2)>
<T2, c, 15>
<START T3>
<T1, d, 20>
<COMMIT T1>
<T3, e, 25>
<COMMIT T2>
<END CKPT>
* FALLA AQUÍ

```

#### i) “¿Hasta dónde debe leer el log?” (2 pts)

Como se encuentra  $\langle \text{END CKPT} \rangle$  leyendo de atrás hacia adelante, el recovery **solo necesita leer hasta el inicio de  $\langle \text{START CKPT (T1, T2)} \rangle$** . No es necesario ir más atrás porque todo lo anterior al checkpoint ya está garantizado en disco.

#### ii) ¿Qué acciones se deshacen? (3 pts)

Leyendo de atrás hacia adelante desde la falla hasta  $\langle \text{START CKPT} \rangle$ :

- $\langle \text{COMMIT } T2 \rangle \rightarrow$  T2 se marca como realizada.
- $\langle T3, e, 25 \rangle \rightarrow$  T3 **no** tiene COMMIT ni ABORT  $\rightarrow$  **se deshace**:  $e := 25$ .
- $\langle \text{COMMIT } T1 \rangle \rightarrow$  T1 se marca como realizada.
- $\langle T1, d, 20 \rangle \rightarrow$  T1 ya marcada como realizada  $\rightarrow$  no se deshace.
- $\langle T2, c, 15 \rangle \rightarrow$  T2 ya marcada como realizada  $\rightarrow$  no se deshace.
- Se llega a  $\langle \text{START CKPT (T1, T2)} \rangle \rightarrow$  se detiene el recovery.

**Conclusión:** solo se deshace la escritura de  $e$  realizada por T3, ya que nunca confirmó. T1 y T2 quedan con sus valores finales porque ambas hicieron commit.

## Pregunta 2 - EEDD, Almacenamiento e Índices

$T(a,b,c,d)$  con 1.500.000 tuplas,  $B$  tuplas/página,  $P$  punteros/página,  $M$  páginas en memoria.

### a) Hash Clustered sobre a (5 pts)

SELECT b, c FROM T WHERE a = 742

En un índice **Hash Clustered**, la búsqueda por igualdad calcula  $hash(742)$  para encontrar directamente el bucket que **contiene la tupla misma** por ser **clustered**.

$$\text{Costo de I/O} = 1$$

Corresponde al caso ideal de una función de hash sin colisiones excesivas: se accede a 1 página (el bucket) que contiene la tupla buscada. En el peor caso, si hubiera overflow pages para ese bucket, el costo sería  $1 + (\text{número de overflow de páginas a recorrer})$ .

### b) B+ Unclustered sobre b (5 pts)

SELECT a, c FROM T WHERE b > 50

Es una consulta de **rango**. En un índice B+ Unclustered, el índice retorna un **puntero** a la página con la tupla, no la tupla misma.

- Tuplas que satisfacen la condición:  $n = 1,500,000 \times \frac{1}{10} = 150,000$  tuplas
- Costo de bajar por el árbol hasta la primera hoja relevante:  $h$  (altura del árbol)
- Al ser unclustered, cada tupla puede estar en una página distinta del disco

$$\text{Costo de I/O} = h + 150,000$$

En el peor caso, cada una de las 150.000 tuplas requiere un acceso a disco independiente, porque al ser unclustered no hay garantía de que estén agrupadas en páginas contiguas.

### c) Verdadero o Falso (5 pts)

- I) Un índice Hash es eficiente para consultas de rango como WHERE a > 100    **F**  
*Los índices Hash están optimizados para búsquedas de igualdad; la función de hash no preserva orden, por lo que no se puede usar para acotar rangos sin recorrer todo el índice.*
- II) En un índice Clustered, el índice retorna la tupla misma    **V**  
*Por definición, el retorno de un índice Clustered es la tupla completa, no un puntero.*

- III) Un índice B+ Unclustered puede requerir hasta  $n$  accesos a disco para  $n$  tuplas distintas en páginas distintas    **V**  
*Como índice retorna sólo un puntero, si las  $n$  tuplas están repartidas en  $n$  páginas diferentes, se necesita un I/O extra por cada una de hasta  $n$  veces.*
- IV) Un índice Hash Clustered tiene costo 1 en el caso ideal para búsquedas de igualdad    **V**  
*En el caso ideal (sin colisiones que generen overflow), se accede directamente al bucket/página que contiene la tupla: costo = 1.*
- v) Los índices Hash usan listas ligadas para resolver problemas de colisiones    **V**  
*El método de resolución de colisiones visto en clases es **encadenamiento (chaining)**: cada colisión agrega un nodo a una lista ligada en el mismo casillero. En índices de disco esto se traduce en **overflow pages** encadenadas.*

## Pregunta 3 - Evaluación de Consultas

### a) Block Nested Loop Join, $M = 10$ (6 pts)

R: 100 páginas, 1.000 tuplas      S: 400 páginas, 2.000 tuplas

**Fórmula:**  $\text{Costo}(R) + \left\lceil \frac{\text{Páginas}(R)}{\text{Buffer} - 2} \right\rceil \cdot \text{Costo}(S)$

i) Usando  $R$  como outer:

$$\text{Costo} = p_R + \left\lceil \frac{p_R}{M - 2} \right\rceil \cdot p_S = 100 + \left\lceil \frac{100}{8} \right\rceil \cdot 400 = 100 + 13 \cdot 400 = \boxed{5,300 \text{ I/Os}}$$

Usando  $S$  como outer (para comparar):

$$\text{Costo} = p_S + \left\lceil \frac{p_S}{M - 2} \right\rceil \cdot p_R = 400 + \left\lceil \frac{400}{8} \right\rceil \cdot 100 = 400 + 50 \cdot 100 = 5,400 \text{ I/Os}$$

ii) “Conviene  $R$  como outer o  $S$  como outer?”

**Conviene usar  $R$  (la relación más pequeña) como outer:**  $5.300 \text{ I/Os} < 5.400 \text{ I/Os}$ .

El costo está dominado por el término  $\lceil p_{\text{outer}} / (M - 2) \rceil \cdot p_{\text{inner}}$ . Al poner la relación más pequeña como outer se necesitan **menos pasadas completas** sobre la relación inner, ya que  $\lceil p_R / (M - 2) \rceil = 13 < \lceil p_S / (M - 2) \rceil = 50$ . Minimizar el número de vueltas sobre la tabla más grande reduce el costo total.

### b) External Merge Sort: relación $R$ con $P$ páginas, $M$ páginas de buffer (4 pts)

i) Fase 0 – Creación de runs iniciales (2 pts)

Se cargan  $M$  páginas de  $R$  al buffer, se ordenan en memoria con un algoritmo de sorting interno, y se escriben de vuelta a disco como un run ordenado. Se repite hasta procesar toda la relación. Se generan  $\lceil P/M \rceil$  runs iniciales, cada uno de tamaño  $M$  páginas (excepto posiblemente el último).

ii) Fases posteriores – Merge (2 pts)

En cada fase se toman grupos de  $(M - 1)$  runs, reservando 1 página de buffer como output y  $M - 1$  páginas como input (una por run). Se hace merge de los  $M - 1$  runs comparando los elementos al frente de cada buffer de entrada y escribiendo el menor al output; al llenarse, se vuelca a disco. El resultado es un run  $(M - 1)$  veces más grande que se materializa en disco. Se repite hasta quedar con un solo run completamente ordenado.

---

**c) Nested Loop Join (6 pts)****i) Definición y costo (2 pts)**

El Nested Loop Join es la implementación más directa: para cada tupla de  $R$  se recorre la relación  $S$  completa buscando coincidencias con el predicado, además de leer  $R$  entera una vez.

$$\text{Costo en I/O} = \text{Costo}(R) + \text{Tuplas}(R) \cdot \text{Costo}(S) = p_R + n_R \cdot p_S$$

**ii) Ejemplo con  $p_R = 500$ ,  $p_S = 800$ ,  $n_R = 10,000$  (2 pts)**

$$\text{Costo} = 500 + 10,000 \cdot 800 = 500 + 8,000,000 = \boxed{8,000,500 \text{ I/Os}}$$

**iii) “Por qué es tan costoso?” (2 pts)**

Porque por **cada tupla individual** de  $R$  (no por cada página) se vuelve a leer la relación  $S$  completa desde disco. Se desperdicia el buffer: solo se usa para mantener una tupla de  $R$  y recorrer  $S$ , en vez de aprovechar el espacio disponible en memoria para cargar muchas tuplas de  $R$  a la vez. El término  $n_R \cdot p_S$  domina el costo total y crece muy rápido cuando ambas relaciones son grandes, haciendo el algoritmo impracticable para tablas de tamaño real (más de 8 millones de I/Os en el ejemplo).



## Pregunta 4 - ORM, NoSQL y Text Search

### a) TF-IDF de “aprender” (5 pts)

- D1: *aprender base de datos es aprender a estructurar datos* → 9 palabras
- D2: *una base de datos guarda datos de forma eficiente* → 9 palabras
- D3: *aprender a programar es aprender a resolver problemas* → 9 palabras

**Paso 1 – Term Frequency (TF):**

$$TF_{D1}(\text{aprender}) = \frac{2}{9} = 0,222 \quad TF_{D2}(\text{aprender}) = \frac{0}{9} = 0 \quad TF_{D3}(\text{aprender}) = \frac{2}{9} = 0,222$$

**Paso 2 – Inverse Document Frequency (IDF):**

$$IDF(t) = \log \left( \frac{\text{número de documentos}}{\text{número de documentos con } t} \right)$$

“aprender” aparece en 2 documentos (D1 y D3) de un total de 3:

$$IDF(\text{aprender}) = \log(3/2) = 0,176$$

**Paso 3 – TF-IDF = TF × IDF:**

| Documento | TF    | IDF   | TF-IDF                                |
|-----------|-------|-------|---------------------------------------|
| D1        | 0.222 | 0.176 | $0,222 \times 0,176 = \mathbf{0,039}$ |
| D2        | 0     | 0.176 | $0 \times 0,176 = \mathbf{0}$         |
| D3        | 0.222 | 0.176 | $0,222 \times 0,176 = \mathbf{0,039}$ |

### b) Categorías de bases de datos NoSQL (5 pts)

| Categoría   | Modelo de datos                                                                                                            | Ejemplo          | Caso de uso ideal                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Clave-Valor | Pares (key, value) sin estructura interna definida por el sistema; el valor es opaco.                                      | Redis, DynamoDB  | Cachés, sesiones de usuario, datos de acceso ultra-rápido por clave (carritos de compra, contadores).                                  |
| Documentos  | Documentos semi-estructurados (JSON/BSON), con campos anidados y arreglos; cada documento puede tener un esquema distinto. | MongoDB, CouchDB | Catálogos de productos, perfiles de usuario, contenido con estructura variable que cambia con frecuencia.                              |
| Grafos      | Nodos y relaciones (aristas) con propiedades; optimizado para recorrer conexiones.                                         | Neo4j            | Redes sociales, sistemas de recomendación, detección de fraude: casos donde las relaciones entre entidades son el dato más importante. |

### c) Verdadero o Falso (5 pts)

- I) MongoDB almacena documentos en formato BSON (similar a JSON) **V**  
*BSON es la representación binaria de JSON usada internamente por MongoDB, agregando tipos adicionales (fechas, binarios, etc.)*
- II) El TF de un término aumenta cuando ese término aparece en muchos documentos del corpus **F**  
*El TF mide la frecuencia del término **dentro de un solo documento** ( $FD(t) = \text{veces que aparece } t \text{ en } D$ ), no su distribución entre documentos. Eso lo mide el IDF.*
- III) El IDF de un término es alto cuando ese término aparece en pocos documentos **V**  
 *$IDF(t) = \log(N/\text{docs\_con\_}t)$ ; si el término aparece en pocos documentos, el denominador es chico y el logaritmo es grande.*
- IV) Las BD relacionales mantienen la consistencia y las documentales no **V**  
*Tradicionalmente las BD documentales priorizan disponibilidad/escalabilidad (teorema CAP) sobre consistencia estricta tipo ACID completo. Las BD relacionales aseguran consistencia*
- V) En MongoDB, los documentos de una misma colección deben tener exactamente el

mismo esquema    **F**

*Una característica central de MongoDB es el esquema flexible: distintos documentos de una misma colección pueden tener campos diferentes.*

## Bonus - 5 puntos

### a) Análisis de afirmaciones (2 pts)

- I) La de-identificación garantiza que los datos anonimizados no pueden ser re-identificados **F**

*Como muestra el caso del gobernador de Massachusetts, la de-identificación (enmascarar nombres) no garantiza protección; con datos auxiliares (cuasi-identificadores) se puede re-identificar a las personas mediante ataques de asociación.*

- II) Un ataque de diferenciación puede revelar información de un individuo ejecutando dos consultas de agregación cuidadosamente diseñadas **V**

*Es el ejemplo visto en clases:  $SUM(edad)$  sobre toda la tabla vs.  $SUM(edad)$  excluyendo a una persona específica permite despejar la edad de esa persona por diferencia.*

- III) El k-anonimato asegura que cada combinación de cuasi-identificadores aparece al menos  $k$  veces en la tabla **V**

*Es la definición formal exacta:  $T$  satisface k-anonimato si y solo si cada grupo de CIds aparece al menos  $k$  veces en  $T$ .*

**Respuesta correcta: B) Solo II y III son verdaderas**

### b) Verificación de 2-anonimato (2 pts)

| Fecha Nac. | Sexo | Código Postal | Enfermedad   |
|------------|------|---------------|--------------|
| 1990       | M    | 750**         | Diabetes     |
| 1990       | M    | 750**         | Hipertensión |
| 1985       | F    | 720**         | Diabetes     |
| 1985       | F    | 720**         | Asma         |

Cuasi-identificadores: {Fecha Nac., Sexo, Código Postal}. Aplicando la definición formal:

- Grupo (1990, M, 750\*\*): 2 registros
- Grupo (1985, F, 720\*\*): 2 registros

**Sí, satisface 2-anonimato.** Cada combinación distinta de cuasi-identificadores aparece exactamente **2 veces** en la tabla ( $\geq k = 2$ ), por lo que ningún individuo es distinguible de al menos otro 1 individuo dentro de su grupo de cuasi-identificadores.

---

**c) Ley 21.719 (1 pto)**

- I) La ley aplica únicamente a empresas privadas, no a entidades públicas   **F**  
*La ley aplica tanto a personas naturales como jurídicas, incluyendo entidades públicas y privadas que operen en Chile.*
- II) El titular de datos tiene derecho a solicitar la eliminación de sus datos cuando ya no sean necesarios   **V**  
*Es el “Derecho de supresión”, uno de los 6 derechos fundamentales que otorga la ley.*
- III) Los datos sensibles incluyen información sobre salud, religión y orientación sexual   **V**  
*Coincide exactamente con la definición de datos sensibles vista en clases.*

**Respuesta correcta: B) Solo II y III son verdaderas**